

第七套题解

目录

T1 回文串 · T2 交错四元组 · T3 文件判定 · T4 树与叶子

T1 回文串

部分分 (30 分, $|s| \leq 3$): 枚举排列与子串

字符最多只有三个, 可以直接尝试所有重排结果。

先把字符串排序, 再用 `next_permutation` 依次生成不同排列。对每个排列, 枚举非空区间 $[l, r]$, 用两个指针从区间两端向中间检查字符是否相同。如果始终相同, 这个区间就是一个回文子串。

统计每个排列的回文子串数, 取最大值。单个字符也要统计; 两个内容相同但位置不同的子串分别计数, 不能去重。

设字符串长度为 L , 不同排列数为 P , 时间复杂度上界为 $O(PL^3)$, 空间复杂度为 $O(L)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    string s;
    cin >> s;
    sort(s.begin(), s.end());
    int n = (int)s.size();
    long long ans = 0;
    do {
        long long count = 0;
        for (int l = 0; l < n; l++) {
            for (int r = l; r < n; r++) {
                bool ok = true;
                int i = l, j = r;
                while (i < j) {
                    if (s[i] != s[j]) {
                        ok = false;
                        break;
                    }
                    i++;
                    j--;
                }
                if (ok) {
                    count++;
                }
            }
        }
        ans = max(ans, count);
    } while (next_permutation(s.begin(), s.end()));
    cout << ans << '\n';
    return 0;
}
```

满分做法：把相同字符放在一起

一个回文子串的首尾字符一定相同。假设某个字符出现 c 次，从这 c 个位置中选择满足 $l \leq r$ 的两个端点，共有

$$1 + 2 + \cdots + c = \frac{c(c+1)}{2}$$

种选择。每对子串端点只能确定一个区间，因此以这个字符为首尾的回文子串，最多有这么多个。

如果把这个字符的所有出现位置排成一个连续块，那么块内的每个子串都是回文，恰好有 $c(c+1)/2$ 个。把不同字符各自排成一个块后，每种字符的上界都能同时达到，所以这就是最优排列。

因此只需统计每个字母的出现次数 cnt_i ，计算

$$ans = \sum_{i=0}^{25} \frac{cnt_i(cnt_i + 1)}{2}.$$

不用真正生成排列，也不用逐个判断回文。计数乘法与答案都应使用 `long long`，因为同一个字符出现 10^6 次时，答案会超过 32 位整数范围。

时间复杂度为 $O(|s|)$ ；计数数组空间为 $O(26)$ ，下面的代码保存输入字符串，总空间复杂度为 $O(|s|)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    string s;
    cin >> s;
    long long cnt[26] = {};
    for (int i = 0; i < (int)s.size(); i++) {
        cnt[s[i] - 'a']++;
    }
    long long ans = 0;
    for (int i = 0; i < 26; i++) {
        // 将相同字符放在一起，该块贡献 1+2+...+cnt[i]。
        ans += cnt[i] * (cnt[i] + 1) / 2;
    }
    cout << ans << '\n';
    return 0;
}
```

T2 交错四元组

部分分（累计 20 分， $n \leq 20$ ）：四重枚举

按下标递增枚举 $i < j < k < l$ ，然后检查

$$a_i < a_k < a_j < a_l.$$

满足条件就把答案加一。四个位置的顺序和四个数的大小顺序不同，尤其是中间两项要求 $a_k < a_j$ 。所有数值比较都必须严格，相等时不能计入。

时间复杂度为 $O(n^4)$ ，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
int a[2005];
int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    long long ans = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) {
            for (int k = j + 1; k <= n; k++) {
                for (int l = k + 1; l <= n; l++) {
                    if (a[i] < a[k] && a[k] < a[j] && a[j] < a[l]) {
                        ans++;
                    }
                }
            }
        }
    }
    cout << ans << '\n';
    return 0;
}
```

部分分（累计 50 分， $n \leq 300$ ）：预处理右端点

固定第二个位置 j 。对于第三个位置 k ，最后一个位置只需满足 $l > k$ 且 $a_l > a_j$ ，因此可以提前统计它的数量。

从右向左计算 $\text{greaterRight}[k]$ ，表示 k 右侧大于 a_j 的元素个数：

$$\text{greaterRight}[k] = \text{greaterRight}[k + 1] + [a_{k+1} > a_j].$$

方括号表示条件成立时取 1，否则取 0。

接着枚举 $i < j$ 和 $k > j$ 。只要 $a_i < a_k < a_j$ ，就将 $\text{greaterRight}[k]$ 加入答案，不再逐个枚举 l 。每个前三位置对应的合法右端点都被一次计入，因此与四重枚举得到的结果相同。

时间复杂度为 $O(n^3)$ ，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

int a[2005], greaterRight[2005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    long long ans = 0;
    for (int j = 2; j <= n - 2; j++) {
        greaterRight[n] = 0;
        for (int k = n - 1; k > j; k--) {
            greaterRight[k] = greaterRight[k + 1];
            if (a[k + 1] > a[j]) {
                greaterRight[k]++;
            }
        }
        for (int i = 1; i < j; i++) {
            for (int k = j + 1; k < n; k++) {
                if (a[i] < a[k] && a[k] < a[j]) {
                    ans += greaterRight[k];
                }
            }
        }
    }
    cout << ans << '\n';
    return 0;
}
```

满分做法：固定中间两项，统计两侧数量

固定中间两个位置 j, k ，并先检查 $a_k < a_j$ 。此时，左端 i 只需满足 $i < j$ 且 $a_i < a_k$ ；右端 l 只需满足 $l > k$ 且 $a_l > a_j$ 。

两边的选择互不影响。如果左侧有 L 个合法位置，右侧有 R 个合法位置，这一对中间位置就贡献 $L \times R$ 个四元组。

用 $\text{leftLess}[j][k]$ 记录左侧的合法位置数。固定 k ，从小到大枚举 j ，可以得到

$$\text{leftLess}[j][k] = \text{leftLess}[j-1][k] + [a_{j-1} < a_k].$$

右侧不必再开二维数组。固定 j 后，从右向左枚举 k ，用 rightGreater 维护严格在 k 右侧且大于 a_j 的元素数。

每次先判断 $a_k < a_j$ ：成立时累加 $\text{leftLess}[j][k] * \text{rightGreater}$ 。随后，如果 $a_k > a_j$ ，就把当前位置加入 rightGreater ，供下一个更小的 k 使用。先统计、后加入，可以始终保证右端位置严格大于 k 。

每个合法四元组都有唯一的中间位置对 (j, k) ；固定它们后，两侧合法位置的所有组合都满足要求，所以不会重复或遗漏。答案可能超过 32 位范围，应使用 `long long`。

时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n^2)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

int a[2005], leftLess[2005][2005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    // leftLess[j][k]: 下标小于 j、数值小于 a[k] 的元素数。
    for (int k = 1; k <= n; k++) {
        for (int j = 2; j < k; j++) {
            leftLess[j][k] = leftLess[j - 1][k];
            if (a[j - 1] < a[k]) {
                leftLess[j][k]++;
            }
        }
    }
    long long ans = 0;
    for (int j = 2; j <= n - 2; j++) {
        long long rightGreater = 0;
        for (int k = n; k > j; k--) {
            if (a[k] < a[j]) {
                ans += leftLess[j][k] * rightGreater;
            }
            // 更新放在计数之后，保证第四个位置严格大于 k。
            if (a[k] > a[j]) {
                rightGreater++;
            }
        }
    }
    cout << ans << '\n';
    return 0;
}
```

T3 文件判定

以下三组特殊性质各独立占 20 分。

部分分（独立 20 分）：同时满足性质 1、2、3

根目录唯一且名称正确，所有名称都正确，目录只有三级。在这些保证下，每个题目对应一条“根目录到题目”的关系和一条“题目到代码文件”的关系，只需判断题目是否恰好有四个。

因此读完所有关系后，检查关系数 n 是否等于 8 即可。这个判断依赖本档的特殊保证；一般数据中，8 条关系也可能有错误命名或错误层次。

设名称最大长度为 L ，每组时间复杂度为 $O(nL)$ ，额外空间复杂度为 $O(L)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        for (int i = 1; i <= n; i++) {
            string a, b;
            cin >> a >> b;
        }
        // 只适用于性质 1、2、3 同时成立的子任务。
        if (n == 8) {
            cout << "yes\n";
        } else {
            cout << "no\n";
        }
    }
    return 0;
}
```

部分分（独立 20 分）：同时满足性质 1、3

本档保证根目录唯一且名称正确，并且只有三级，但题目名称、文件名称和数量仍可能有误。

先根据包含关系找到根，再检查它是否恰好有四个直接孩子。每个题目名称必须全是英文字母；每个题目恰好有一个孩子，并且孩子名称必须等于“题目名称 + .cpp”。同时要求总关系数为 8，排除多余条目。

这些检查覆盖了除根命名以外的所有规则，而根命名由题目保证，因此不需要再检查根名的字母、数字格式。

设名称最大长度为 L ，每组时间复杂度为 $O(n^2L)$ ，空间复杂度为 $O(nL)$ 。

参考代码

完整代码见下文“部分分综合做法”。

部分分（独立 20 分）：满足性质 2

本档所有名称都符合要求，只可能在根目录个数或第二层目录个数上出错。因此先寻找根，要求根唯一，再确认它恰好包含四个题目。

下面的综合程序还会检查每题唯一的代码文件。对本档而言，这些名称和文件关系已经有保证，多做检查不会改变结果；而根数或题目数错误的输入仍会被拒绝。

设名称最大长度为 L ，使用综合程序时，每组时间复杂度为 $O(n^2L)$ ，空间复杂度为 $O(nL)$ 。

参考代码

与上一档使用同一程序，完整代码见下文“部分分综合做法”。

部分分综合做法

三组特殊性质都保证根名称合法。因此可以共用一份程序：完整检查关系数、根的唯一性、题目数量、题目名和代码文件名，只省略根名称的格式检查。

找根时，检查一个出现在父项位置的名称是否曾作为子项出现。没有父亲的名称才是根候选；同一个根会出现在多条关系中，只能按名称算作一个候选。

题目顺序和输入行顺序没有关系，必须扫描全部关系。检查四个题目及各自的代码后，恰好使用全部八条关系，不能再有额外层次或文件。

设名称最大长度为 L ，每组时间复杂度为 $O(n^2L)$ ，空间复杂度为 $O(nL)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

string a[11], b[11];
int n;

bool isLetter(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
}

bool check() {
    if (n != 8) {
        return false;
    }
    string root = "";
    for (int i = 1; i <= n; i++) {
        bool hasParent = false;
        for (int j = 1; j <= n; j++) {
            if (a[i] == b[j]) {
                hasParent = true;
            }
        }
        if (!hasParent) {
            if (root != "" && root != a[i]) {
                return false;
            }
            root = a[i];
        }
    }
    return true;
}
```

```

    }
    root = a[i];
}
}
// 部分分保证根名称合法；这里仅判断根是否存在。
if (root == "") {
    return false;
}
int tasks = 0;
for (int i = 1; i <= n; i++) {
    if (a[i] != root) {
        continue;
    }
    tasks++;
    string task = b[i];
    for (int j = 0; j < (int)task.size(); j++) {
        if (!isLetter(task[j])) {
            return false;
        }
    }
    int files = 0;
    for (int j = 1; j <= n; j++) {
        if (a[j] == task) {
            files++;
            if (b[j] != task + ".cpp") {
                return false;
            }
        }
    }
    if (files != 1) {
        return false;
    }
}
// 四条根边 + 四条代码文件边已经占满全部八条关系。
return tasks == 4;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        cin >> n;
        for (int i = 1; i <= n; i++) {
            cin >> a[i] >> b[i];
        }
        if (check()) {
            cout << "yes\n";
        } else {
            cout << "no\n";
        }
    }
    return 0;
}

```

满分做法：逐项核对名称与包含关系

合法目录一定有四条“根目录到题目”的关系和四条“题目到代码文件”的关系，总共八条。因此如果 $n \neq 8$ ，可以直接判断为不合法； $n = 8$ 时还要继续检查具体内容。

用两个字符串数组保存每条关系的父项和子项。对每个出现过的父项，扫描所有子项名称：如果它从未作为子项出现，就是一个根候选。根必须存在，而且所有根候选必须是同一个名称。不能把第一行的父项直接当作根，因为输入顺序任意。

根名称分两段检查：先读一段非空英文字母，再读一段非空数字；字母和数字都不能缺少，进入数字部分后也不能再出现字母。例如 BJ01 合法，123、ABC、A2B 都不合法。

随后扫描根的直接孩子，要求恰好四个，并逐一检查：题目名称是非空纯字母串；题目恰好有一个孩子；孩子名称逐字符等于“题目名称 + .cpp”。大小写必须一致，只检查扩展名是不够的。

为什么不用继续检查第四层？四个题目及各自的代码文件已经占满八条关系。输入关系不重复，同名条目又不会位于不同位置；当上述检查全部通过时，所有关系都已用完，不可能再有多余孩子或游离条目。

每组最多十条关系，直接扫描即可，不需要额外建立复杂树结构。设名称最大长度为 L ，每组时间复杂度为 $O(n^2L)$ ，空间复杂度为 $O(nL)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

string a[11], b[11];
int n;

bool isLetter(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
}

bool rootNameOK(const string &s) {
    int i = 0;
    while (i < (int)s.size() && isLetter(s[i])) {
        i++;
    }
    if (i == 0 || i == (int)s.size()) {
        return false;
    }
    while (i < (int)s.size()) {
        if (s[i] < '0' || s[i] > '9') {
            return false;
        }
        i++;
    }
    return true;
}

bool check() {
    if (n != 8) {
        return false;
    }
    string root = "";
    for (int i = 1; i <= n; i++) {
        bool hasParent = false;
        for (int j = 1; j <= n; j++) {
            if (a[i] == b[j]) {
                hasParent = true;
            }
        }
        if (!hasParent) {
            if (root != "" && root != a[i]) {
                return false;
            }
            root = a[i];
        }
    }
    if (!rootNameOK(root)) {
        return false;
    }
    int tasks = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] != root) {
```

```
        continue;
    }
    tasks++;
    string task = b[i];
    for (int j = 0; j < (int)task.size(); j++) {
        if (!isLetter(task[j])) {
            return false;
        }
    }
    int files = 0;
    for (int j = 1; j <= n; j++) {
        if (a[j] == task) {
            files++;
            if (b[j] != task + ".cpp") {
                return false;
            }
        }
    }
    if (files != 1) {
        return false;
    }
}
// 四条根边 + 四条代码文件边已经占满全部八条关系。
return tasks == 4;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        cin >> n;
        for (int i = 1; i <= n; i++) {
            cin >> a[i] >> b[i];
        }
        if (check()) {
            cout << "yes\n";
        } else {
            cout << "no\n";
        }
    }
    return 0;
}
```

T4 树与叶子

小规模两档按累计分计算；链与最优断边唯一两档各独立占 10 分。

部分分（累计 10 分， $n \leq 100$ ）：枚举断边

本档只有一组数据。逐条枚举要删除的边，分别从两个端点出发做 BFS，统计断边后两部分的叶子数量。

不必真的从邻接表删除边，只需在遍历时跳过这条边。计算一个点的新度数时，先取原度数；如果它被删边的端点，就减一。新度数不超过 1 时计为叶子，所以断边产生的孤立点也要统计。

得到两边叶子数后，计算差的绝对值。出现更小的差值时，将方案数重置为 1；差值相等时，方案数加一。

每组时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

vector<int> g[100005];
int edgeU[100005], edgeV[100005], que[100005];
bool seen[100005];
int n, bannedU, bannedV;

int countLeaves(int start) {
    for (int i = 1; i <= n; i++) {
        seen[i] = false;
    }
    int head = 0, tail = 1, result = 0;
    que[0] = start;
    seen[start] = true;
    while (head < tail) {
        int u = que[head++];
        int degree = (int)g[u].size();
        if (u == bannedU || u == bannedV) {
            degree--;
        }
        if (degree <= 1) {
            result++;
        }
        for (int j = 0; j < (int)g[u].size(); j++) {
            int v = g[u][j];
            if ((u == bannedU && v == bannedV) ||
                (u == bannedV && v == bannedU)) {
                continue;
            }
            if (!seen[v]) {
                seen[v] = true;
                que[tail++] = v;
            }
        }
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
```

```

cin >> t;
while (t--) {
    cin >> n;
    for (int i = 1; i <= n; i++) {
        g[i].clear();
    }
    for (int i = 1; i < n; i++) {
        cin >> edgeU[i] >> edgeV[i];
        g[edgeU[i]].push_back(edgeV[i]);
        g[edgeV[i]].push_back(edgeU[i]);
    }
    int ans = n, ways = 0;
    for (int i = 1; i < n; i++) {
        bannedU = edgeU[i];
        bannedV = edgeV[i];
        int left = countLeaves(bannedU);
        int right = countLeaves(bannedV);
        int difference = abs(left - right);
        if (difference < ans) {
            ans = difference;
            ways = 1;
        } else if (difference == ans) {
            ways++;
        }
    }
    cout << ans << ' ' << ways << '\n';
}
return 0;
}

```

部分分（累计 20 分， $n \leq 1000$ ）：继续按定义统计

仍然可以使用上一节的断边枚举。每次 BFS 都要清空访问标记；处理新的一组树之前，清空邻接表，并重新初始化最小差值与方案数。

算法仍是逐条尝试全部边，再按删除后的度数数叶子；输入允许多组，并不改变每一组的处理方法。

每组时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

参考代码

代码与上一节“部分分（累计 10 分， $n \leq 100$ ）：枚举断边”相同，不重复列出。

部分分（独立 10 分）：树是一条链

一条链至少有两个点时，叶子数为 2；只有一个点时，叶子数为 1。因此只需按 n 分类。

$n = 2$ 时，删除唯一的边得到两个孤点，输出 0 1。 $n = 3$ 时，两条边都把链分成一个孤点和一条两点链，输出 1 2。

$n \geq 4$ 时，删除两条端边会产生一个孤点，两侧叶子数为 1 和 2。其余 $n - 3$ 条边都让两侧至少有两个点，两侧叶子数均为 2，因此输出 0 和 $n - 3$ 。

时间复杂度为 $O(n)$ ，用于读入全部边；额外空间复杂度为 $O(1)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        for (int i = 1; i < n; i++) {
            int u, v;
            cin >> u >> v;
        }
        if (n == 2) {
            cout << "0 1\n";
        } else if (n == 3) {
            cout << "1 2\n";
        } else {
            cout << 0 << ' ' << n - 3 << '\n';
        }
    }
    return 0;
}
```

部分分（独立 10 分）：最优断边唯一

这条性质直接确定第二个输出为 1，但不能确定最小叶子数差值，也不能确定最优边在哪里。

可以使用本题满分做法的子树叶子计数与端点修正，逐边求出差值，再取最小值。唯一性保证只省去并列方案的统计，寻找最小差值的过程仍然需要完成。

每组时间复杂度为 $O(n)$ ，空间复杂度为 $O(n)$ 。

参考代码

完整代码见本题满分题解中的“满分做法：原叶子计数与端点修正”。该程序也能处理本档，不重复列出。

部分分综合做法

先检查所有点的度数是否都不超过 2。输入已经保证是一棵树，所以满足这个条件时就是一条链，直接使用上面的分类公式。

其他树采用断边枚举与 BFS。这就同时处理了小规模的一般树和大规模的链，而不必让长链也执行平方级的重算。

每组链的时间复杂度为 $O(n)$ ，一般树为 $O(n^2)$ ；空间复杂度为 $O(n)$ 。

参考代码

```

#include <bits/stdc++.h>
using namespace std;

vector<int> g[100005];
int edgeU[100005], edgeV[100005], que[100005];
bool seen[100005];
int n, bannedU, bannedV;

int countLeaves(int start) {
    for (int i = 1; i <= n; i++) {
        seen[i] = false;
    }
    int head = 0, tail = 1, result = 0;
    que[0] = start;
    seen[start] = true;
    while (head < tail) {
        int u = que[head++];
        int degree = (int)g[u].size();
        if (u == bannedU || u == bannedV) {
            degree--;
        }
        if (degree <= 1) {
            result++;
        }
        for (int j = 0; j < (int)g[u].size(); j++) {
            int v = g[u][j];
            if ((u == bannedU && v == bannedV) ||
                (u == bannedV && v == bannedU)) {
                continue;
            }
            if (!seen[v]) {
                seen[v] = true;
                que[tail++] = v;
            }
        }
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        cin >> n;
        for (int i = 1; i <= n; i++) {
            g[i].clear();
        }
        for (int i = 1; i < n; i++) {
            cin >> edgeU[i] >> edgeV[i];
            g[edgeU[i]].push_back(edgeV[i]);
            g[edgeV[i]].push_back(edgeU[i]);
        }
        bool chain = true;
        for (int i = 1; i <= n; i++) {
            if (g[i].size() > 2) {
                chain = false;
            }
        }
        if (chain) {
            if (n == 2) {
                cout << "0 1\n";
            } else if (n == 3) {
                cout << "1 2\n";
            } else {
                cout << 0 << ' ' << n - 3 << '\n';
            }
            continue;
        }
    }
}

```

```

    }
    int ans = n, ways = 0;
    for (int i = 1; i < n; i++) {
        bannedU = edgeU[i];
        bannedV = edgeV[i];
        int left = countLeaves(bannedU);
        int right = countLeaves(bannedV);
        int difference = abs(left - right);
        if (difference < ans) {
            ans = difference;
            ways = 1;
        } else if (difference == ans) {
            ways++;
        }
    }
    cout << ans << ' ' << ways << '\n';
}
return 0;
}

```

满分做法：原叶子计数与端点修正

删除一条边，只会使它的两个端点度数各减一，其他点的叶子身份完全不变。

对一个端点来说：原度数为 1，删除后变成 0，它原本就是叶子，仍然算叶子；原度数为 2，删除后变成 1，会新增一个叶子；原度数至少为 3，删除后仍不是叶子。

因此，只要知道一条边两侧各有多少个原叶子，再补上原度数为 2 的端点即可。

任选点 1 作为遍历根。令 $leaf[v]$ 表示 v 的子树中，原树度数恰好为 1 的点数；令 $L = leaf[1]$ ，即整棵树的原叶子数。对父子边 (u, v) ，删除后两侧的叶子数分别为

$$L_1 = leaf[v] + [deg(v) = 2],$$

$$L_2 = L - leaf[v] + [deg(u) = 2].$$

方括号表示条件成立时取 1，否则取 0。其中一侧恰好是 v 的子树，另一侧是它的补集，两个端点的新增叶子分别加在各自一侧。原度数为 1 的端点即使变成孤点，也已经计入原叶子数，不能再重复加一。

计算 $leaf$ 时，先用迭代遍历得到一个父亲先于孩子出现的顺序，并记录每个点的父亲。初始化每个点的计数为“它的原度数是否等于 1”，再逆序扫描，把每个点的计数累加给父亲。

最后枚举所有非根节点 v ，就枚举了全部父子边。按公式计算 $|L_1 - L_2|$ ，维护最小值和相应边数。找到严格更小的值时，方案数重置为 1；遇到相同值时再加一。

这里统计的是无根树中度数为 1 的原叶子，不是“有根树中没有孩子的点”；遍历根本身也可能是原叶子。迭代遍历还能避免长链上的递归栈问题。

每组时间复杂度为 $O(n)$ ，空间复杂度为 $O(n)$ ；多组的总时间复杂度为 $O(\sum n)$ 。

参考代码

```

#include <bits/stdc++.h>
using namespace std;

vector<int> g[100005];
int parentNode[100005], orderNode[100005], leaf[100005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        for (int i = 1; i <= n; i++) {
            g[i].clear();
            parentNode[i] = 0;
        }
        for (int i = 1; i < n; i++) {
            int u, v;
            cin >> u >> v;
            g[u].push_back(v);
            g[v].push_back(u);
        }
        // 先记录父亲在前的遍历顺序, 避免长链递归爆栈。
        int count = 1;
        orderNode[1] = 1;
        for (int i = 1; i <= count; i++) {
            int u = orderNode[i];
            leaf[u] = 0;
            if (g[u].size() == 1) {
                leaf[u] = 1;
            }
            for (int j = 0; j < (int)g[u].size(); j++) {
                int v = g[u][j];
                if (v != parentNode[u]) {
                    parentNode[v] = u;
                    orderNode[++count] = v;
                }
            }
        }
        // leaf[u] 只统计原树中度为 1 的叶子。
        for (int i = n; i >= 2; i--) {
            int u = orderNode[i];
            leaf[parentNode[u]] += leaf[u];
        }
        int ans = n, ways = 0;
        for (int v = 2; v <= n; v++) {
            int u = parentNode[v];
            int left = leaf[v];
            int right = leaf[1] - leaf[v];
            if (g[v].size() == 2) {
                left++;
            }
            if (g[u].size() == 2) {
                right++;
            }
            int difference = abs(left - right);
            if (difference < ans) {
                ans = difference;
                ways = 1;
            } else if (difference == ans) {
                ways++;
            }
        }
        cout << ans << ' ' << ways << '\n';
    }
    return 0;
}

```